PHASE 1A: EVENT TICKETING PLATFORM

Section 2 — Week-by-Week Execution Breakdown

April 4 – April 24, 2026 | 3.5 Weeks | 105–122 Hours

Section Overview

This section provides the complete day-by-day execution plan for Phase 1A, organized into three focused weeks. Each day includes morning, afternoon, and evening work blocks with specific tasks, implementation notes, and a concrete deliverable.

Week	Dates	Focus	Milestones	Project Hrs	Total
Week 1	Apr 4–10	Backend Scaffold + Event Service + Auth + DB Schema	Services scaffolded, DB migrated, Event CRUD live, Docker running	22–24	35–40
Week 2	Apr 11–17	Booking State Machine + Stripe + RabbitMQ + Redis Locking	State machine tested, Stripe checkout + webhooks, no double-bookings	22–24	35–40
Week 3	Apr 18–24	Frontend + Testing + Deployment + Polish	All 7 pages live, 80%+ coverage, k6 load tests, deployed URL, CI/CD	18–20	28–32
TOTAL	3.5 wks	Full-Stack Ticketing Platform	Enterprise-grade, deployed, documented	62–68	105–122

WEEK 1: April 4–10

Days 1–7 • 35–40 Hours

Theme: Foundation & Structure

Week 1 Goals

- Scaffold all 7 backend modules — structure, DTOs, entities, repository interfaces. No business logic yet.
- Implement full database schema with Flyway migrations (9 migrations total).
- **Event CRUD fully working:** POST, GET, PUT, DELETE endpoints with full validation and authorization.
- **Auth service reused from Phase 0:** JWT + RBAC copied in, adapted with the ORGANIZER role added.
- **Docker Compose:** Running PostgreSQL, Redis, RabbitMQ, and Spring Boot in a single command.
- **Next.js initialized:** Tailwind configured, basic layout, home page rendering with real data.
- All code committed using conventional commits, organized into feature branches.

Track	Hours
Project (building)	22–24 hrs
Fundamentals (System Design reading)	8–10 hrs
Practices (TDD, docs, Git)	5–6 hrs
Total	**35–40 hrs**

DAY 1 — Saturday, April 4

Theme: Project Initialization + Database Schema

8 hrs planned hours

Morning *(2 hrs)*

- **System Design reading:** Read "Designing Data-Intensive Applications" Ch. 1 (Reliable, Scalable, Maintainable Systems). Skim for big ideas; take 5 bullet notes.
- **Database schema design:** Sketch the full DB schema on paper or Excalidraw — all 9 tables, relationships, and which columns need indexes. Take a screenshot. This is ERD v1.

Afternoon *(5 hrs)*

Skim Spring State Machine docs overview: https://docs.spring.io/spring statemachine/docs/current/reference/ — just get the vocabulary (states, transitions, guards, actions). Don't implement yet

Spring Boot Project Setup

- Generate the project at `start.spring.io` with the following dependencies: Spring Web, Spring Data JPA, Spring Security, Spring AMQP, Spring Data Redis, Flyway Migration, PostgreSQL Driver, Lombok, Validation, Actuator, Testcontainers (test scope).

- Manually add to `pom.xml`: `stripe-java (23.3.0)`, `spring-statemachine-core (3.2.0)`, `jjwt (0.11.5)`, `qrgen (2.6.0)`.
- Project settings: name `ticketing-platform`, group `com.ticketing`, Java 17.

Package Structure & Placeholder Classes

- Create the full package structure (see Section 4). Create empty placeholder classes with `@Service`, `@Repository`, `@RestController` annotations — no methods yet. Goal: the project compiles cleanly.

Flyway Migrations

- Write all migration files under `src/main/resources/db/migration/`:
  - `V1__create_users_table.sql`
  - `V2__create_venues_and_categories.sql`
  - `V3__create_events_table.sql`
  - `V4__create_ticket_tiers.sql`
  - `V5__create_bookings_and_tickets.sql`
  - `V6__create_payments_and_refunds.sql`
  - `V7__create_indexes.sql`

JPA Entities

- Write all JPA entities: User, Event, Category, Venue, TicketTier, Booking, Ticket, Payment, Refund. Use Lombok `@Data`, `@Builder`, `@NoArgsConstructor`, `@AllArgsConstructor`. Add `@Version Long version` on the Booking entity for optimistic locking.

Docker Compose

- Write `docker-compose.yml` with PostgreSQL 15, Redis 7, RabbitMQ 3.12 (with management plugin), and a Spring Boot app stub. Start infrastructure with `docker-compose up -d postgres redis rabbitmq`. Verify all three are running with `docker ps`.
- Write `application.yml` reading all datasource, Redis, and RabbitMQ connection properties from environment variables. Write `application-local.yml` with hardcoded local dev values. Run the app — it should start and connect to the database without errors.

Evening *(1 hr)* @Test @DisplayName ("should create event successfully"))

- **TDD setup:** Create `EventServiceTest`. Write 3 failing test stubs with `@Test` and `@DisplayName` annotations. Do not implement yet — these will be made green on Day 2.
- Git: `git init`, add `.gitignore`, first commit: `chore: initialize spring boot project with full package structure`. Create GitHub repo and push.
- Add `README.md` skeleton (project name, setup instructions placeholder, architecture section placeholder). Commit: `docs: add readme skeleton`.

Deliverable: Running Docker Compose (3 services), Spring Boot app starts and connects to DB, full package structure created, all entities defined, Flyway runs migrations on startup. GitHub repo created.

DAY 2 — Sunday, April 5

Theme: Event Service + Auth Integration

7 hrs planned hours

Morning *(1.5 hrs)*

- **Auth service integration:** Copy Phase 0 auth service code into `com.ticketing.user`. Adapt: change package names, add `ORGANIZER` to the Role enum (was USER/ADMIN before), ensure `JwtService`, `UserDetailsServiceImpl`, and `SecurityConfig` are adapted for this app.

  Add @PreAuthorize("hasRole('ORGANIZER')") import (don't use yet, just verify it's wired)

Afternoon *(4.5 hrs)*

Event Repository

- Write `EventRepository` with three custom queries:
  - `findByStatusAndStartDateAfter(EventStatus status, LocalDateTime date)` — for filtering upcoming published events.
  - `findByStatusAndCategoryIdAndVenueCity(EventStatus status, Long categoryId, String city, Pageable pageable)` — for filtered, paginated list.
  - `findByIdWithDetails(Long id)` — an `@Query` using JOIN FETCH on organizer and category to eliminate N+1 loading.

DTOs

- Write `CreateEventRequest` and `UpdateEventRequest` with validation annotations: `@NotBlank`, `@NotNull`, `@Future` for startDate, `@Min(1)` for capacity.
- Write `EventResponse` DTO with `@Builder` for readable test assertions.

EventService — TDD Approach

- Write ALL EventService tests first (Red), then implement EventService to make them pass (Green):
  - `createEvent_withValidData_shouldReturnEvent()`
  - `createEvent_withPastDate_shouldThrowValidationException()`
  - `getEvent_withNonExistentId_shouldThrowNotFoundException()`
  - `updateEvent_asNonOrganizer_shouldThrowForbiddenException()`
  - `updateEvent_whenPublished_shouldChangeState()`

  Run: all RED. Now implement EventService to make them GREEN
- Implement `EventService` with these methods:
  - `createEvent(CreateEventRequest req, Long organizerId)` — persists Event with `status=DRAFT`, creates TicketTiers from request.
  - `getEventById(Long id)` — uses `findByIdWithDetails` to avoid N+1.

- `getEvents(EventFilterRequest filter, Pageable pageable)` — filtered and paginated list.
- `updateEvent(Long id, UpdateEventRequest req, Long organizerId)` — validates ownership before update.
- `deleteEvent(Long id, Long organizerId)` — only allowed if `status=DRAFT`.
- `publishEvent(Long id, Long organizerId)` — DRAFT → PUBLISHED transition, validates all required fields are present.

EventController & Exception Handling

- Implement `EventController` with all 5 endpoints. Add `@Validated` and `@PreAuthorize` annotations. Use `ResponseEntity<ApiResponse<EventResponse>>` as the response wrapper pattern.
- Write `GlobalExceptionHandler` (`@ControllerAdvice`) handling: `EntityNotFoundException`, `AccessDeniedException`, `ValidationException`, `MethodArgumentNotValidException`.

Evening *(1 hr)*

- Test all 5 Event endpoints with Postman or HTTPie. Create `ticketing-platform.postman_collection.json` and commit it to the repo.
- Refactor: ensure no method exceeds 20 lines, all variable names are descriptive, and no raw strings are used for status values.
- Git commit: `feat: implement event crud with organizer authorization`. Push.

Deliverable: All 5 Event endpoints working, JWT auth working, Postman collection passes all requests, EventService tests: 7/7 passing.

DAY 3 — Monday, April 6

Theme: Venue, Category, Search + Schema Polish

5 hrs planned hours

Morning *(1 hr)*

- **Research:** Read about `@EntityGraph` in Spring Data JPA (Baeldung reference). Understand the difference between FETCH and LOAD types. Take notes — you will apply this today.

Afternoon *(3 hrs)*

Venue & Category Services

- Implement `VenueService` and `CategoryService` as straightforward CRUD services. Write 3 tests each (Red) before implementing (Green).

Event Search Service

- Implement `EventSearchService` with a custom `@Query` that accepts optional filter parameters (`query`, `categoryId`, `city`), restricts to `status=PUBLISHED` and future events, and supports pagination.
- Add endpoint: `GET /api/search/events?q=&category=&city=&page=&size=`.

Schema Refinements

- Refine Flyway migrations — add missing columns discovered during implementation (e.g., `waitlist_enabled boolean` and `dynamic_pricing_enabled boolean` on events). Create `V8__add_event_features.sql`.
- Write `CategoryController` and `VenueController`. Add seed data migration `V9__seed_data.sql` with 5 categories (Music, Sports, Comedy, Theater, Festival) and 3 venues.

Evening *(1 hr)*

- Fix any failing tests. Ensure `./mvnw test` runs clean — all green.
- Write `TicketTierService` — manages availability per tier. Critical method: `getAvailableCount(Long tierId)`, which will be used extensively in the booking flow. Write tests for it.

Deliverable: Category/Venue CRUD working, Search endpoint working with filters, seed data loaded, ./mvnw test passes all tests.

DAY 4 — Tuesday, April 7

Theme: Next.js Setup + Home Page + Event Listing

5 hrs planned hours

Morning *(1 hr)*

- **Frontend architecture planning:** Sketch the 7 pages, the components each page needs, and which API calls each page makes. Record this in `frontend/ARCHITECTURE.md`.

Afternoon *(3.5 hrs)*

Next.js Project Initialization

- Initialize a Next.js 14 project inside `frontend/` using `create-next-app@latest` with flags: `--typescript`, `--tailwind`, `--eslint`, `--app`, `--src-dir`.
- Install additional packages: `@tanstack/react-query`, `axios`, `swr`, `zustand`, `react-hook-form`, `zod`, `@stripe/stripe-js`, `@stripe/react-stripe-js`, `lucide-react`, `date-fns`.
- Set up `src/lib/api.ts` with an Axios base configuration and request interceptors that automatically attach JWT tokens from the auth store.

Layout Components

- Build `Navbar.tsx`: logo, search bar, login/register links, cart icon with count badge.
- Build `Footer.tsx` and `RootLayout.tsx`.

Home Page

- Build `app/page.tsx` with the following sections:
  - Hero section: large gradient banner, "Find Your Next Experience" headline, search input with city and date filters, "Search Events" CTA button.
  - Category filter row: horizontal scrollable row of category pill buttons (Music, Sports, Comedy, Theater, Festival).
  - "Upcoming Events" section: responsive grid of `EventCard` components.
  - `EventCard.tsx`: cover image placeholder (gradient), event title, date badge, location icon + city, price "From $XX", "Book Now" CTA button — clean card design with hover shadow.
  - Fetch events from `GET /api/events?status=PUBLISHED` using React Query with a 5-minute stale time.

Evening *(1 hr)*

- Wire up search functionality on the home page — category pill click triggers a new API call with updated query params.
- Git commit: `feat: add nextjs frontend with home page and event listing`. Push.

Deliverable: Next.js app running on port 3000, home page renders with real data from Spring Boot, category filters work.

DAY 5 — Wednesday, April 8
Theme: Inventory Service + Redis Cache Layer

5 hrs
planned hours

Morning *(1 hr)*

SETNX, EXPIRE, DEL, GETSET patterns:

- **Research:** Read the Redis documentation on distributed lock patterns. Key decision: use the atomic `SET key value NX EX 300` command instead of the non-atomic `SETNX + EXPIRE` pattern, which has a race condition if the process crashes between the two commands.

Afternoon *(3.5 hrs)*

InventoryService

- Implement `InventoryService` with these core methods, backed by `RedisTemplate<String, Integer>`:
  - `getAvailableCount(Long tierId)` — reads from Redis cache first (key: `inventory:tier:{tierId}:available`), falls back to the DB on a cache miss.
  - `reserveSeat(Long tierId, Long userId, int quantity)` — atomically checks availability and decrements the count in Redis.

  - `releaseSeat(Long tierId, int quantity)` — atomically increments the count in Redis.
  - `commitReservation(Long tierId, int quantity)` — finalizes the reservation (no-op when using DB as source of truth).
- On application startup, load all tier availability from the DB into Redis to warm the cache.

CacheService for Events

- Implement event caching using `@Cacheable`, `@CacheEvict` annotations with a `RedisCacheManager` configuration:
  - On `getEventById`: check Redis cache key `event:{id}`; on miss, fetch from DB and store with a 10-minute TTL.
  - On `updateEvent`: evict cache key `event:{id}`.
  - On `publishEvent`: evict all event list caches.

RedisConfig

- Write `RedisConfig.java`: configure `RedisTemplate<String, Object>` with Jackson serialization; configure `RedisCacheManager` with a default 10-minute TTL and per-cache TTL overrides. Define cache name constants: `EVENT_CACHE`, `EVENT_LIST_CACHE`, `TIER_AVAILABILITY_CACHE`.

InventoryService Tests (Testcontainers)

- Write 4 tests for `InventoryService` using a real Redis container:
  - `reserveSeat_whenSufficientInventory_shouldDecrementCount()`
  - `reserveSeat_whenInsufficientInventory_shouldReturnFalse()`
  - `releaseSeat_shouldIncrementCount()`
  - `concurrentReservations_shouldNotOversell()` — 10 threads, 10 tickets, only 5 available → exactly 5 succeed.

Evening *(1 hr)*

RabbitMQ Configuration (Infrastructure Only)

- Write `RabbitMQConfig.java` (this is infrastructure config only — queues will be consumed in Week 2). Declare:
  - Exchanges: `booking.exchange` (topic), `notification.exchange` (direct).
  - Queues: `booking.confirmation.queue`, `email.notification.queue`, `ticket.generation.queue`, plus a Dead Letter Queue (DLQ) for each.
  - Bind queues to exchanges with routing keys.
- Git commit: `feat: implement inventory service with redis and rabbitmq config`.

Deliverable: InventoryService tests passing (including concurrency test), Redis event caching working, RabbitMQ configured with all queues and DLQs.

DAY 6 — Thursday, April 9 *Theme: Integration Testing + N+1 Fixes + EXPLAIN ANALYZE*	5 hrs planned hours

Morning *(1 hr)*

- **Research:** Read the vladmihalcea.com article on the N+1 query problem — understand WHY it happens, not just the fix. Write down 3 specific N+1 scenarios in your current codebase where it will occur.

Afternoon *(3.5 hrs)*

Eliminating N+1 Queries

- Enable SQL logging: set `spring.jpa.show-sql=true` and `spring.jpa.properties.hibernate.format_sql=true`. Count the queries generated when fetching 10 events — if you see 30+ queries, that is the N+1 problem.
- Fix `findAll()` → add `@EntityGraph(attributePaths = {"organizer", "category", "venue"})` on the repository method.
- Fix event list for organizer → use `JOIN FETCH` in a `@Query` instead of lazy loading.
- Goal: fetching 10 events should result in 1–3 queries maximum, not one per relationship.

Testcontainers Integration Test

- Set up a `EventIntegrationTest` class using `@SpringBootTest` + `@Testcontainers`. Spin up a `PostgreSQLContainer` and a `GenericContainer` for Redis. Use `@DynamicPropertySource` to inject container connection details into Spring properties.
- Write integration tests for the full event lifecycle: `createEvent → getEvent → updateEvent → publishEvent` and `searchEvents` with filters.

Look at the code in v.1 pdf

Database Query Analysis

- Run `EXPLAIN ANALYZE` in PostgreSQL on the event search query. Learn to read the output: look for "Seq Scan" on large tables (bad) vs "Index Scan" (good). Verify that indexes from `V7__create_indexes.sql` are being used.

EventControllerTest

- Write `EventControllerTest` using `@WebMvcTest`: test all 5 endpoints for happy path and error cases, mock `EventService` with Mockito, and test that the ORGANIZER role is required for create/update/delete operations.

Evening *(1 hr)*

- Fix any flaky tests. Run the full test suite 3 times — it must be deterministic (never fails randomly).

- Document N+1 findings in the README under a "Performance" section. Show the before/after query count. Git commit: `perf: fix n+1 queries in event and category loading`.

Deliverable: Zero N+1 queries in the Event module (verified with SQL logging), integration tests passing, EXPLAIN ANALYZE output documented in README.

DAY 7 — Friday, April 10 *Theme: Week 1 Cleanup + System Design Introduction*	6 hrs planned hours

Morning *(2 hrs)*

- **System Design reading:** Find "Design Ticketmaster" resources. Read `https://systemdesign.one/ticketmaster-system-design/`. Draw a quick architecture diagram showing: CDN, Load Balancer, API servers, DB (primary/replica), Redis cluster, RabbitMQ.
- **Key insight to internalize:** "What is the hardest problem in a ticketing system?" Answer: race conditions during high-concurrency ticket purchase. Keep this insight — it directly motivates the Redis locking work in Week 2.

Afternoon *(3 hrs)*

Week 1 Code Review

we may use codeQL or any other Tool

- Go through every file written this week. Apply the self-code-review checklist from Section 13. Fix any violations before moving forward.

Docker Compose — Full Configuration

- Add `pgAdmin` service for DB browsing during development.
- Add `redis-commander` service for Redis key inspection.
- RabbitMQ Management UI is already exposed at port `15672`.
- Configure health checks on all services and document all ports in the README.

Polish Event Detail API

- Update the Event Detail API response to include all fields in a single query using `@EntityGraph`: event info (all fields), venue info (name, address, city, capacity), ticket tier info (tier name, price, available count, total capacity), and organizer info (name, profile).

Evening *(1 hr)*

- Verify Week 1 checkpoint: go through each success criteria checkbox below. Note anything missing as "carry-forward" to Monday with a specific completion plan.
- Git: commit all Week 1 work, push. Add a Week 1 summary comment to the GitHub PR describing what you built, what you learned, and what was hardest.

Deliverable: Week 1 checkpoint fully met. Clean codebase, all tests green, Docker Compose starts all services with one command.

Check the v.1 PDF for the correct checklist

WEEK 1 CHECKPOINT — Sunday, April 10

- ☑ [object Object][object Object]
- ☑ 9 Flyway migrations run successfully, DB schema matches the ERD
- ☑ [object Object][object Object][object Object][object Object][object Object][object Object][object Object][object Object][object Object][object Object][object Object]
- ☑ Authentication working: JWT login, role-based access (USER/ORGANIZER)
- ☑ Next.js home page renders with real data from Spring Boot
- ☑ [object Object][object Object]
- ☑ EventService tests: 8+ tests, all green
- ☑ [object Object][object Object][object Object]

Week 1 Resources

- Spring Boot project generator: `https://start.spring.io`
- Flyway migrations docs: `https://flywaydb.org/documentation/concepts/migrations`
- Spring Data JPA @EntityGraph: `https://www.baeldung.com/spring-data-jpa-named-entity-graphs`
- Redis Distributed Locks: `https://redis.io/docs/manual/patterns/distributed-locks/`
- N+1 Query deep dive: `https://vladmihalcea.com/n-plus-1-query-problem/`
- Designing Data-Intensive Applications (DDIA) — Chapters 1 and 2
- Testcontainers for Spring: `https://testcontainers.com/guides/testing-spring-boot-rest-api-using-testcontainers/`
- Design Ticketmaster reference: `https://systemdesign.one/ticketmaster-system-design/`

WEEK 2: April 11–17

Days 8–14 • 35–40 Hours

Theme: The Hard Stuff — State Machine, Stripe, RabbitMQ, Redis Locking

Week 2 Goals

- Full booking state machine implemented with all transitions, guards, and actions — tested.
- Stripe checkout session creation and webhook handling working end-to-end.
- RabbitMQ queues processing booking confirmations, email notifications, and QR code generation.
- Redis distributed locking preventing double-bookings — verified with a 100-thread concurrent test.
- Frontend: Event detail page, cart with 5-minute countdown timer, and checkout flow.
- "Design Ticketmaster" — 45-minute timed system design practice.

Track	Hours
Project (building)	22–24 hrs
Fundamentals (System Design)	8–10 hrs
Practices (TDD, docs, Git)	5–6 hrs
Total	**35–40 hrs**

DAY 8 — Saturday, April 11

Theme: Booking State Machine — The Heart of the System

8 hrs planned hours

Morning *(2 hrs)*

- **Spring State Machine research:** Read the Baeldung Spring State Machine tutorial — focus on the "Events and Transitions" and "Actions and Guards" sections. Understand: `StateMachineConfig` defines states and transitions; `@WithStateMachine` wires the machine to a service; Guards are conditions that block transitions; Actions are side effects triggered by transitions.
- **State machine diagram:** Draw the complete state machine diagram on paper — both the Booking and Event state machines. Every arrow = a transition. Every node = a state. This drawing is your implementation target before writing any code.

Guards: conditions on transitions ("can only go PAYMENT_PENDING from RESERVED")

Afternoon *(5 hrs)*

Actions: side effects on transitions ("start 5-min timer on RESERVED entry")

Enum Definitions

- Define `BookingState`:
  - `AVAILABLE, RESERVED, PAYMENT_PENDING, CONFIRMED, ATTENDED, EXPIRED, RELEASED, PAYMENT_FAILED, REFUND_REQUESTED, REFUND_APPROVED, REFUND_DENIED`

- Define `BookingEvent` (triggers):
  - `ADD_TO_CART, PROCEED_TO_CHECKOUT, PAYMENT_SUCCESS, PAYMENT_FAILURE, TIMER_EXPIRED, CHECK_IN, REQUEST_REFUND, APPROVE_REFUND, DENY_REFUND`
- Define `EventStatus`:
  - `DRAFT, PUBLISHED, SALES_OPEN, SALES_CLOSED, COMPLETED, ARCHIVED`

BookingStateMachineConfig

- Implement `BookingStateMachineConfig.java` annotated with `@Configuration` and `@EnableStateMachine`. Configure states via `StateMachineStateConfigurer` and transitions via `StateMachineTransitionConfigurer`. Each external transition declares source, target, event, guard, and action.
- **Key transitions to implement:**
  - `AVAILABLE → RESERVED` on `ADD_TO_CART`: guarded by `seatsAvailableGuard`, action `startReservationTimerAction`.
  - `RESERVED → PAYMENT_PENDING` on `PROCEED_TO_CHECKOUT` (no guard, no action).
  - `PAYMENT_PENDING → CONFIRMED` on `PAYMENT_SUCCESS`: action `confirmBookingAction` + `generateQRCodeAction`.
  - `PAYMENT_PENDING → PAYMENT_FAILED` on `PAYMENT_FAILURE`: action `releaseSeatsAction`.
  - `RESERVED → EXPIRED` on `TIMER_EXPIRED`: action `releaseSeatsAction`.
  - `CONFIRMED → REFUND_REQUESTED` on `REQUEST_REFUND`.
  - `REFUND_REQUESTED → REFUND_APPROVED` on `APPROVE_REFUND`: action `releaseSeatsAction`.
  - `REFUND_REQUESTED → REFUND_DENIED` on `DENY_REFUND`.

BookingStateMachineTest — TDD

- Write `BookingStateMachineTest` first (all Red), then implement guards and actions to make them Green:
  - `transition_fromAvailableToReserved_shouldLockSeatsInRedis()`
  - `transition_fromReservedToExpired_afterFiveMinutes_shouldReleaseSeats()`
  - `transition_fromPaymentPendingToConfirmed_shouldGenerateQRCode()`
  - `transition_fromReservedToExpired_whenSeatsAvailable_shouldNotTransition()`
  - `invalidTransition_fromConfirmedToReserved_shouldThrowException()`

BookingService.reserveTickets()

- Implement `BookingService.reserveTickets()` as a `@Transactional` method with these steps in order:

1. Validate the event exists and its status is SALES_OPEN.
2. Validate the tier has availability via `InventoryService.getAvailableCount()`.
3. Acquire Redis distributed lock: `SET seat:lock:{tierId} {userId} NX EX 300` (atomic, 5-minute TTL).
4. Double-check availability while holding the lock (prevents TOCTOU race condition).

5. Decrement inventory via `InventoryService.reserveSeat()`.
6. Create Booking entity with `state=RESERVED`.
7. Create Ticket entities (one per quantity requested).
8. Store cart data in Redis hash: `HSET cart:{userId} bookingId, tierId, quantity, eventId`.
9. Schedule a timer task for 5-minute expiration.
10. Return `BookingResponse`. — If any step 3–9 fails: release the lock, re-increment inventory, and throw a descriptive exception.

Evening *(1 hr)*

- **Reservation expiry timer:** Create `ReservationExpirationJob` as a `@Component` with a `@Scheduled(fixedDelay = 30000)` method. Every 30 seconds, the job queries for all bookings where `state=RESERVED` and `expires_at < now`. For each expired booking, it sends the `TIMER_EXPIRED` event to the state machine, which triggers `releaseSeatsAction` to return tickets to inventory.
- Git commit: `feat: implement booking state machine with all transitions and guards`.

Deliverable: BookingStateMachine tests: 5/5 passing. reserveTickets() implemented. Timer expiration job running every 30 seconds.

DAY 9 — Sunday, April 12

Theme: Stripe Integration

7 hrs planned hours

Resources within v.1 pdf page 14

Morning *(1.5 hrs)*

- **Stripe research:** Read the Stripe Checkout quickstart (Java section), the webhooks guide (specifically signature verification), and the testing reference (test card numbers). Create a free Stripe account and obtain test API keys (`sk_test_...` and `pk_test_...`).

Stripe has 2 modes (this is the key), When you use Stripe, everything revolves around:

☑ Test Mode (what YOU need)

Afternoon *(4.5 hrs)*

Stripe Configuration

- Create `StripeConfig.java` with `@Configuration`. Inject the secret key from `${stripe.secret-key}` via `@Value`. In `@PostConstruct`, assign `Stripe.apiKey = secretKey`.
- Add to `application.yml`:
  - `stripe.secret-key: ${STRIPE_SECRET_KEY}`
  - `stripe.webhook-secret: ${STRIPE_WEBHOOK_SECRET}`
  - `stripe.success-url` (pointing to the confirmation page with `{bookingId}` and `{CHECKOUT_SESSION_ID}` placeholders)
  - `stripe.cancel-url` (pointing to `/cart`)

PaymentService.createCheckoutSession()

- Implement `PaymentService.createCheckoutSession(Long bookingId, Long userId)` with this flow:

11. Fetch booking and validate it belongs to the requesting user. Throw `BookingNotFoundException` if not found.
12. Validate booking is in `RESERVED` state. Throw `InvalidStateTransitionException` otherwise.
13. Transition booking to `PAYMENT_PENDING` via the state machine (prevents a second checkout attempt while the Stripe session is open).
14. Build line items from `booking.getTickets()` — one `SessionCreateParams.LineItem` per ticket with `PriceData` containing currency, unit amount (price × 100 for cents), and product name.
15. Call `Session.create(SessionCreateParams)` with mode PAYMENT, success URL, cancel URL, all line items, a 30-minute `expiresAt`, and metadata containing `bookingId` and `userId`.
16. Save a `Payment` entity with `stripe_session_id` and `status=PENDING`.
17. Return `CheckoutSessionResponse` containing the Stripe session URL.

StripeWebhookController

- Create `StripeWebhookController` at `POST /api/webhooks/stripe`. Accept the raw request body (as String) and the `Stripe-Signature` header.
- Verify the signature using `Webhook.constructEvent(payload, sigHeader, webhookSecret)`. Return HTTP 400 on `SignatureVerificationException`.
- **Idempotency:** Before processing, check `processedEventRepository.existsByStripeEventId()`. If already processed, return `200 OK` immediately to avoid duplicate processing.
- Switch on `event.getType()`:
  - `checkout.session.completed` → call `handlePaymentSuccess(event)`: extract `bookingId` from metadata, update Payment entity (`status=COMPLETED`, `stripe_payment_id`), send `PAYMENT_SUCCESS` to state machine → CONFIRMED, generate QR codes, publish `BookingConfirmedEvent` to RabbitMQ.
  - `checkout.session.expired` → call `handlePaymentExpired(event)`: send `PAYMENT_FAILURE` to state machine, releasing the seats.
- Save the Stripe event ID to `ProcessedStripeEvent` table after successful processing. Return `200 OK`.

Local Webhook Testing

- Use Stripe CLI for local webhook testing: `stripe listen --forward-to localhost:8080/api/webhooks/stripe`. Copy the webhook signing secret printed to the console and set it as the `STRIPE_WEBHOOK_SECRET` environment variable.

Evening *(1 hr)*

- Write `PaymentServiceTest`:

- `createCheckoutSession_forValidBooking_shouldReturnStripeUrl()`
- `handleWebhook_withInvalidSignature_shouldReturn400()`
- `handlePaymentSuccess_shouldTransitionBookingToConfirmed()`
- Mock all Stripe SDK calls with Mockito — never call real Stripe in unit tests.

- Git commit: `feat: implement stripe checkout and webhook handling`.

> **Deliverable:** Stripe checkout session created from API call. Test payment with card 4242 4242 4242 4242 succeeds. Webhook receives the success event and transitions the booking to CONFIRMED.

DAY 10 — Monday, April 13

Theme: RabbitMQ Message Queues + Email Notifications

5 hrs planned hours

Morning *(1 hr)*

- **Research:** Read RabbitMQ AMQP concepts — specifically topic exchanges vs direct exchanges and routing keys. Read the Dead Letter Queue (DLQ) pattern documentation to understand what happens when a message fails processing repeatedly.

Afternoon *(3.5 hrs)*

Message Publishers

messagingpublishers

- Implement `BookingEventPublisher` with three publishing methods, all using `RabbitTemplate` and automatically JSON-serialized via `Jackson2JsonMessageConverter`:
  - `publishBookingConfirmation(BookingConfirmedEvent event)` → sends to `booking.exchange` with routing key `booking.confirmed`.
  - `publishEmailNotification(EmailNotificationEvent event)` → sends to `notification.exchange` with routing key `email.send`.
  - `publishTicketGeneration(TicketGenerationEvent event)` → sends to `booking.exchange` with routing key `ticket.generate`.
- Define message POJO classes: `BookingConfirmedEvent`, `EmailNotificationEvent`, `TicketGenerationEvent`, `WaitlistAvailableEvent` — include all fields consumers will need.

Notification Listeners

- Implement `BookingNotificationListener` with three `@RabbitListener` methods:
  - `handleBookingConfirmation()` on `booking.confirmation.queue`: fetch booking details, generate QR code, send confirmation email.
  - `handleEmailNotification()` on `email.notification.queue`: call `emailService.sendEmail()`.
  - `handleTicketGeneration()` on `ticket.generation.queue`: call `ticketService.generateQrCodes()`.

QR Code Generation

- Add ZXing dependencies: `com.google.zxing:core:3.5.1` and `com.google.zxing:javase:3.5.1`.
- Implement `QRCodeGeneratorUtil`: use `QRCodeWriter` to encode content as `BarcodeFormat.QR_CODE` at 200×200 pixels. Store the result in `tickets.qr_code` as a Base64-encoded string.
- QR content: a JSON object containing `ticketId`, `bookingId`, `eventId`, `seatNumber`, `userId` — signed with the app secret for tamper detection.

Email Service with Mailhog

- Add Mailhog to `docker-compose.yml` (SMTP port `1025`, Web UI port `8025`).
- Configure `JavaMailSender` bean from `application.yml`.
- Implement `sendBookingConfirmation(BookingConfirmedEvent event)` — HTML email with ticket details and the QR code embedded as an inline Base64 image.

Evening *(1 hr)*

- Write `NotificationListenerIntegrationTest` with a real RabbitMQ container (Testcontainers): publish a `BookingConfirmedEvent`, wait using `Awaitility.await().atMost(5, SECONDS).until(...)`, then verify the email was sent (mock `EmailService`) and QR code generated.
- Git commit: `feat: implement rabbitmq message queues and email notifications`.

Deliverable: All 3 RabbitMQ queues processing messages, QR codes generating correctly, email notification integration test passing.

DAY 11 — Tuesday, April 14

Theme: Redis Distributed Locking + Concurrent Booking Tests

5 hrs planned hours

Morning *(1 hr)*

- **Research:** Read about the Redlock algorithm and understand the risk of single-node Redis locks (what happens if the Redis node crashes mid-purchase?). For this project, single-node is acceptable — document this trade-off in the README.

Afternoon *(3.5 hrs)*

DistributedLockService

- Implement `DistributedLockService` with four methods:
  - `acquireLock(String lockKey, String lockValue, long ttlSeconds)` — uses `redisTemplate.opsForValue().setIfAbsent(key, value, Duration.ofSeconds(ttl))`, which maps to the atomic `SET key value NX EX` command.

Read: https://redis.io/docs/manual/patterns/distributed-locks/#the-redlock-algorithm.
Understand the risk of single-node locks (what if Redis node crashes?). For this project, single node is fine — note the production trade-off in your docs

Never use `SETNX + EXPIRE` (two commands = race condition if process crashes between them).

- `releaseLock(String lockKey, String lockValue)` — executes a Lua script atomically: `if redis.call('get', KEYS[1]) == ARGV[1] then return redis.call('del', KEYS[1]) else return 0 end`. This prevents accidentally releasing another thread's lock.
- `extendLock(String lockKey, String lockValue, long ttlSeconds)` — extends the TTL only if the caller still owns the lock (another Lua script check).
- `executeWithLock(String lockKey, long ttlSeconds, Supplier<T> operation)` — acquires the lock, runs the operation inside a try block, and releases the lock in a finally block (ensuring release even on exception). Throws `LockAcquisitionException` if the lock cannot be acquired.

- Use lock key prefix `seat:lock:` and a `UUID.randomUUID().toString()` as the lock value to enable safe ownership-checked release.

Concurrent Booking Test — The Most Important Test

- Write `ConcurrentBookingTest` with `@SpringBootTest` + `@Testcontainers`:
  - Setup: create an event with exactly 50 available tickets.
  - Execute: 100 threads simultaneously call `bookingService.reserveTickets()` using `ExecutorService.newFixedThreadPool(100)`, `AtomicInteger` for success/failure counts, and `CountDownLatch` to synchronize thread completion.
  - Assert: `successCount.get() == 50` and `failCount.get() == 50`. No oversell. No data corruption.
  - This test will fail until the locking implementation is correct. Fix until it passes consistently.

Integration into BookingService

- Integrate `DistributedLockService` into `BookingService.reserveTickets()`. Use lock key `tier:{tierId}:user:{userId}` to prevent the same user from double-clicking.
- Keep the `@Version` field on the Booking entity as a second safety net — if Redis fails, the database-level optimistic locking provides a fallback.
- Handle `OptimisticLockingFailureException` with retry logic: 3 attempts with exponential backoff.

Evening *(1 hr)*

- Document the race condition prevention strategy in the README: diagram showing what happens with 100 concurrent requests, and how Redis lock + optimistic locking provides two independent layers of safety.
- Git commit: `feat: implement redis distributed locking preventing double-bookings with concurrent tests`.

Deliverable: Concurrent booking test passing: 100 users → exactly 50 succeed, 50 fail, zero oversell, zero data corruption.

DAY 12 — Wednesday, April 15

Theme: Multi-Tier Pricing + Refund Logic + Waitlist

5 hrs planned hours

Plan the pricing engine

Morning *(1 hr)*

- **Planning:** On paper, write out all 4 pricing rules (early bird, group, VIP tiers, dynamic) as decision trees. The order of application matters: dynamic pricing multiplies the base price first, then discounts apply on top.

Afternoon *(3.5 hrs)*

PricingEngine — TDD

(all pricing scenarios + edge cases)

- Write 8 test cases for `PricingEngine` first (all Red), then implement to make them Green:
- `calculatePrice(TicketTier tier, int quantity, LocalDate eventDate, int currentSoldCount)` — applies rules in order:
  - **Rule 1 — Dynamic Pricing:** If `currentSoldCount / tier.getCapacity() > 0.80`, apply 1.25× surge multiplier.
  - **Rule 2 — Early Bird:** If `ChronoUnit.DAYS.between(LocalDate.now(), eventDate) >= 30`, apply 0.50× discount.
  - **Rule 3 — Group Discount:** If `quantity >= 5`, apply 0.90× discount.
  - Return `finalPrice.multiply(BigDecimal.valueOf(quantity)).setScale(2, RoundingMode.HALF_UP)`.
- `calculateRefundAmount(Booking booking, LocalDateTime cancelTime)` — three tiers:
  - ≥ 7 days until event → 100% refund (full `booking.getTotalAmount()`).
  - 3–6 days until event → 50% refund.
  - < 3 days until event → no refund (return `BigDecimal.ZERO`).

WaitlistService

- Implement `WaitlistService` using a Redis Sorted Set with timestamp as the score (FIFO ordering):
  - `joinWaitlist(Long eventId, Long tierId, Long userId)` → `ZADD waitlist:{eventId}:{tierId} {System.currentTimeMillis()} {userId}`.
  - `notifyNextInWaitlist(Long eventId, Long tierId)` → `ZPOPMIN waitlist:{eventId}:{tierId} 1` to get the earliest waiter, then publish a `WaitlistAvailableEvent` via RabbitMQ to notify them.
- **Integration:** When `releaseSeat()` is called (by either expiration or cancellation), also call `notifyNextInWaitlist()`.

Refund Flow

Policy

- `POST /api/bookings/{id}/cancel` → validates refund eligibility via `PricingEngine.calculateRefundAmount()` → creates a `Refund` entity with `status=PENDING` → publishes `RefundRequestedEvent`.

◦ Handle partial refunds (50% case):

- `RefundService.processRefund(Long refundId)` → calls `Stripe.refund.create()` with `RefundCreateParams` (setting `paymentIntent`, `amount` in cents, reason `REQUESTED_BY_CUSTOMER`) → updates Payment status → transitions booking to `REFUND_APPROVED` → calls `releaseSeat()` and `notifyNextInWaitlist()`.

Evening *(1 hr)*

- Write `PricingEngineTest` — target 100% branch coverage. Every pricing rule, every edge case, every refund tier must be tested.
- Git commit: `feat: implement pricing engine with early-bird, group discounts, dynamic pricing, and refund logic`.

Deliverable: PricingEngine 8/8 tests passing with 100% branch coverage. Waitlist join and notify-on-release working. Refund flow processes successfully via Stripe.

DAY 13 — Thursday, April 16

Theme: Frontend — Event Detail + Cart + Checkout

5 hrs planned hours

Morning *(1 hr)*

- **Research:** Read React Query docs on optimistic updates. You will use this in the cart: optimistically add to cart and show the item immediately, then rollback the UI if the `POST /api/bookings/reserve` call fails.

https://tanstack.com/query/latest/docs/react/guides/optimistic-updates

Afternoon *(3.5 hrs)*

Event Detail Page (app/events/[id]/page.tsx)

- Full-width cover image with a fallback gradient if no image URL is provided.
- Event info section: large title, date/time row with calendar icon, venue row with map pin icon, expandable description (collapsible if over 300 characters).
- Ticket tier selector: 3 cards side by side (VIP, Gold, Standard) — each showing tier name, price (with crossed-out original if a discount applies), available count badge, and a "Select" button. Show "Sold Out" badge and disable the button when availability is zero.
- Quantity selector: +/- buttons with validation (minimum 1, maximum 10).
- Live price preview: "2 × Gold $45.00 = $90.00" — updates reactively as tier or quantity changes using React state.
- "Add to Cart" button → calls `POST /api/bookings/reserve` → on success, redirects to `/cart`.
- "Join Waitlist" button: shown only when the selected tier is sold out. Calls the waitlist API.

Cart Page (app/cart/page.tsx)

- Cart items list: event thumbnail, event name, date, tier name, quantity, price per ticket, line total.

- **Countdown timer component:** displays MM:SS counting down from 5 minutes. Turns red when under 60 seconds. On reaching zero: show a "Reservation expired" modal with a "Find tickets again" button that clears the cart and redirects to the event page.
- Order total breakdown: subtotal, any applied discount badge, grand total.
- "Proceed to Checkout" CTA → calls `POST /api/bookings/{id}/checkout` → redirects to the Stripe Checkout hosted page.
- "Remove Item" → calls the booking cancel API, which triggers the state machine to release the reservation and return tickets to inventory.

Booking Confirmation Page (app/bookings/[id]/confirmation/page.tsx)

- Success animation: a checkmark appear animation using CSS or Framer Motion.
- Booking reference number: displayed large, with a one-click copy-to-clipboard button.
- Event summary card with all key event details.
- Ticket cards: one card per ticket, each rendering the QR code image from the Base64 string, ticket ID, and seat information.
- "Download Tickets (PDF)" button.
- "Add to Google Calendar" button (generates a Google Calendar link using the event's date and title).
- "Share" button.

Evening *(1 hr)*

- Connect the cart countdown to real API data — verify the timer reads the `expires_at` timestamp from the booking response and counts down correctly, including the expired modal behavior.
- Git commit: `feat: add event detail page, cart with countdown timer, booking confirmation page`.

Deliverable: Full ticket purchase flow works end-to-end in the browser: browse event → select tier → add to cart → countdown visible → proceed to checkout → Stripe test payment → confirmation page with QR codes.

DAY 14 — Friday, April 17

Theme: Week 2 Integration Tests + System Design Practice + Cleanup

6 hrs
planned hours

Morning *(2 hrs)*

- **System Design timed practice — "Design Ticketmaster" (45 minutes strictly):** Draw the full architecture diagram (CDN, Load Balancer, API Gateway, App servers, DB with primary/2 replicas, Redis cluster, RabbitMQ, S3 for ticket PDFs).
- **Capacity estimation:** 10M users, 100K concurrent during peak, 1K events/day, 10M tickets/day. Write 3–5 key design decisions (why Redis locks vs DB locks, why async email via

queue, why PostgreSQL for bookings). Identify the bottleneck: the seat lock acquisition is the critical path. Stop at 45 minutes — review gaps afterwards.

why Redis for locks vs DB locks, why async email via queue vs synchronous, why PostgreSQL (ACID) vs Cassandra for bookings

Afternoon *(3 hrs)*

◦ Bottlenecks: the seat lock is the critical path — identify it

Complete Booking Flow Integration Test

- Write a single comprehensive integration test `completeBookingFlow_fromReserveToConfirmed()` that validates the entire flow:

18. Reserve tickets → assert `booking.getState() == RESERVED`.
19. Create checkout session → assert URL starts with `https://checkout.stripe.com` and booking is now in `PAYMENT_PENDING`.
20. Simulate webhook payment success via `webhookService.handlePaymentSuccess(bookingId, "pi_test_123")`.
21. Verify final state: `booking.getState() == CONFIRMED`, all tickets have non-null `qrCode` values.
22. Verify RabbitMQ message was published (mock `RabbitTemplate`, verify the publish call was made with correct arguments).
23. Verify Redis lock was released: `assertThat(lockService.isLocked("tier:" + tierId)).isFalse()`.

- Run the full test suite: `./mvnw clean test`. Fix any failures. Check coverage: `./mvnw jacoco:report`. Current target: 60%+ overall (80%+ achieved in Week 3).
- Code review all Week 2 code. Fix any Clean Code violations before closing the week.

Evening *(1 hr)*

- Week 2 checkpoint review — go through each success criteria item below.
- Git: squash feature branch commits, write detailed PR description, merge to develop, push.

Deliverable: Complete booking flow integration test passes. "Design Ticketmaster" system design diagram created. Week 2 checkpoint fully met.

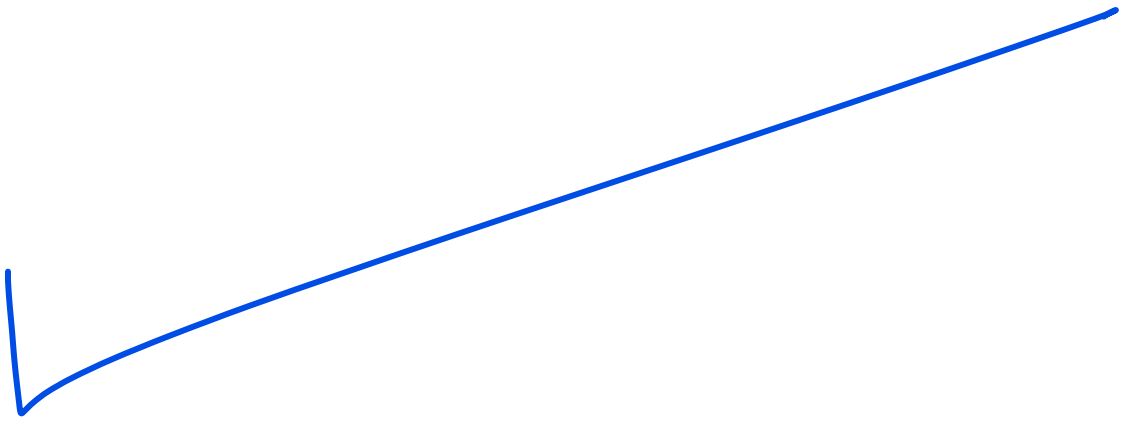
WEEK 2 CHECKPOINT — Friday, April 17

- ☑ Booking state machine: all transitions + guards + actions implemented, 5+ tests, all GREEN
- ☑ Stripe: checkout session creation + webhook handling (success and failure cases)
- ☑ [object Object][object Object][object Object][object Object][object Object][object Object]
- ☑ Redis distributed locking: concurrent booking test with 100 threads passes consistently
- ☑ N+1 queries: zero in both Booking and Event modules (verified with SQL logging)
- ☑ Frontend: Event Detail + Cart (with countdown) + Checkout → Confirmation flow works end-to-end
- ☑ Complete booking flow integration test passes
- RabbitMQ: 3 queues processing, DLQ configured, email notifications sending (Mailhog)

- [x] "Design Ticketmaster" architecture diagram created

Week 2 Resources

- Spring State Machine official docs: `https://docs.spring.io/spring-statemachine/docs/current/reference/`
- Spring State Machine Baeldung tutorial: `https://www.baeldung.com/spring-state-machine`
- Stripe Checkout quickstart (Java): `https://stripe.com/docs/checkout/quickstart?lang=java`
- Stripe webhooks guide: `https://stripe.com/docs/webhooks`
- Stripe test card numbers: `https://stripe.com/docs/testing`
- RabbitMQ Spring AMQP reference: `https://docs.spring.io/spring-amqp/docs/current/reference/html/`
- Redis distributed lock patterns: `https://redis.io/docs/manual/patterns/distributed-locks/`
- Redlock algorithm: `https://redis.io/docs/manual/patterns/distributed-locks/#the-redlock-algorithm`
- Testcontainers Spring Boot guide: `https://testcontainers.com/guides/testing-spring-boot-rest-api-using-testcontainers/`



WEEK 3: April 18–24

Days 15–21 • 28–32 Hours

Theme: Polish, Test, Ship

Week 3 Goals

- Complete all remaining frontend pages: User Dashboard, Organizer Dashboard, Checkout page.
- Achieve 80%+ JaCoCo test coverage overall; 100% on PricingEngine, BookingStateMachine, and RefundService.
- k6 load test: 1,000 concurrent users, P95 < 500ms, zero double-bookings.
- CI/CD pipeline via GitHub Actions: tests on every push, Docker image built, deploy to Railway on main.
- Production deployment: live URL on Railway with health checks passing.
- Complete documentation: Swagger UI, full README with screenshots and diagrams.
- "Design Eventbrite" — 45-minute timed system design practice.

Track	Hours
Project (building)	18–20 hrs
Fundamentals (System Design)	6–7 hrs
Practices (TDD, docs, Git)	4–5 hrs
Total	**28–32 hrs**

DAY 15 — Saturday, April 18

Theme: Remaining Frontend Pages

7 hrs planned hours

Read Mostly

Morning *(1.5 hrs)*

- **Planning:** Design the User Dashboard and Organizer Dashboard on paper. Determine: what data does each display, which API calls are needed, and what actions users can perform from each dashboard.

Afternoon *(4.5 hrs)*

User Dashboard (app/dashboard/page.tsx) — Protected Route

- Redirect unauthenticated users to `/auth/login`.
- Profile card: avatar placeholder, full name, email, member since date.
- Stats row: Total Bookings, Upcoming Events, Total Spent (formatted currency).
- "Upcoming Events" section: list of future bookings rendered as EventCard variants with a "View Tickets" CTA.

- "Past Events" section: attended events with a "Leave Review" button (placeholder for Phase 1B).
- "Booking History" table: date, event name, tier, quantity, total amount, and a color-coded status badge (CONFIRMED in green, EXPIRED in gray, REFUNDED in orange).

Organizer Dashboard (app/organizer/events/page.tsx) — ORGANIZER Role Required

- "Create New Event" button opens a multi-step form modal:
  - Step 1: Basic Info (title, description, category, venue).
  - Step 2: Schedule (start date/time, end date/time, sale open/close dates).
  - Step 3: Ticket Tiers (add VIP/Gold/Standard tiers with price and capacity).
  - Step 4: Review and Publish.
- "My Events" table: event name, status badge, dates, tickets sold/capacity progress bar, revenue total, and action buttons (Edit, Publish, Close Sales, View Attendees).
- Event Analytics card: total revenue chart using recharts (LineChart), tickets sold over time.

Checkout Page (app/checkout/page.tsx)

- Order summary sidebar: event thumbnail, tier name, quantity, price per ticket, subtotal, and grand total.
- Attendee info form: name, email, phone — pre-filled from the logged-in user profile. Validated with react-hook-form + Zod.
- "Complete Purchase" button → calls `POST /api/bookings/{id}/checkout` → on success, receives the Stripe URL → executes `window.location.href = stripeUrl` to redirect to the Stripe hosted payment page.

Loading Skeletons

- Add Tailwind `animate-pulse` loading skeletons to all data-fetching pages. This prevents blank flashes while data loads and provides a better perceived performance experience.

Evening *(1 hr)*

- Test every frontend page manually in the browser. Open Chrome DevTools and test at 375px width for mobile responsiveness.
- Git commit: `feat: complete all 7 frontend pages with user and organizer dashboards`.

> **Deliverable:** All 7 frontend pages implemented and rendering with real data. Mobile responsive. Authentication guard on all protected routes.

DAY 16 — Sunday, April 19

Theme: Test Coverage Push to 80%+

7 hrs planned hours

Morning *(1.5 hrs)*

- Run `./mvnw jacoco:report` and open `target/site/jacoco/index.html`. Identify all modules below 80% coverage. Find every uncovered branch (shown as red lines in the report). List the top 10 most impactful uncovered methods, prioritizing `PricingEngine`, `BookingService`, and `PaymentService` (highest business risk).

Afternoon *(4.5 hrs)*

Missing Unit Tests — BookingService

- Write tests covering all state transition scenarios — both happy path and every error path:
  - `reserveTickets_whenEventNotOpen_shouldThrow()`
  - `reserveTickets_whenQuantityExceedsLimit_shouldThrow()`
  - `cancelBooking_within3Days_shouldRefundZero()`
  - `cancelBooking_between3And7Days_shouldRefund50Percent()`
  - `cancelBooking_moreThan7DaysBefore_shouldRefundFull()`

Missing Integration Tests

- `BookingControllerTest`: all endpoints, happy path + auth failures (no token, wrong role, wrong user).
- `PaymentWebhookTest`: valid signature success, invalid signature rejected (400), duplicate webhook idempotency.
- `WaitlistIntegrationTest`: user joins waitlist, seat released by another user, notification sent and verified.
- `RefundServiceTest`: all 3 refund tiers (full, partial, zero), successful Stripe mock call, correct state machine transition.

Evening *(1 hr)*

- Re-run JaCoCo: `./mvnw jacoco:report`. Must show 80%+ overall. If not, identify the remaining gaps and write those tests before the end of the day.
- Git commit: `test: achieve 80 percent test coverage across all modules`.

Deliverable: JaCoCo report shows 80%+ overall coverage. 100% branch coverage on PricingEngine, BookingStateMachine, and RefundService.

DAY 17 — Monday, April 20

Theme: k6 Load Testing + Performance Tuning

5 hrs planned hours

Morning *(1 hr)*

- Install k6: `brew install k6` or download the binary from `https://k6.io/docs/get-started/installation/`.

Afternoon *(3.5 hrs)*

k6 Load Test Script (load-test.js)

- Write `load-test.js` with the following structure:
- **Scenario — "ticket_rush":** Use the `ramping-vus` executor. Ramp up from 0 to 100 VUs over 30 seconds, spike to 1,000 VUs for 1 minute, then ramp down over 30 seconds.
- **Thresholds:** `http_req_duration` P95 < 500ms; error rate < 1%; a custom `double_bookings` counter must remain at zero.
- **Virtual user function:** Each VU represents a different user (use `__VU` as the user ID). Authenticate, then call `POST /api/bookings/reserve` with `eventId`, `tierId`, `quantity: 1`. Accept both HTTP 200 (success) and HTTP 409 (sold out) as expected responses. Flag anything else as an error.
- Run the test: `k6 run --env BASE_URL=http://localhost:8080 load-test.js`.

Performance Tuning

- **HikariCP pool exhaustion:** If the test shows pool timeouts, increase `spring.datasource.hikari.maximum-pool-size` to 20.
- **Redis timeout:** If Redis operations are timing out under load, increase `spring.data.redis.timeout`.
- **Slow queries:** Run EXPLAIN ANALYZE on the booking insert under load. Add missing indexes if needed.
- **Lock contention:** Ensure the distributed lock TTL is short enough not to block other users for long periods.
- **Target:** P95 response time < 500ms, zero double bookings. Keep tuning until both are met.

Load Test Documentation

- Document results in the README: k6 output screenshot, P50/P95/P99 response times, throughput (requests/sec), error rate, and a SQL verification query confirming zero oversell: `SELECT tier_id, COUNT(*) FROM bookings GROUP BY tier_id HAVING COUNT(*) > [capacity]` (should return no rows).

Evening *(1 hr)*

- Add HikariCP connection pool monitoring to Spring Actuator: expose `/actuator/metrics/hikaricp.connections.active`.
- Git commit: `perf: load test results documented, connection pool tuned`.

Deliverable: k6 load test output showing P95 < 500ms and zero double-bookings. Connection pool tuned and documented in README.

DAY 18 — Tuesday, April 21	5 hrs

Theme: CI/CD + Docker Optimization	planned hours

Morning *(1 hr)*

- **Research:** Read the GitHub Actions documentation on workflow YAML syntax — specifically triggers, jobs, services, steps, and environment variables.

Check v.1 PDF CI.yml Code page 30

Afternoon *(3.5 hrs)*

.github/workflows/ci.yml — CI Pipeline

Note: Keep the full CI/CD YAML configuration. Below is the complete workflow structure.

```
name: CI Pipeline on:   push:     branches: [main, develop]   pull_request:     branches: [main]
jobs:   test:     runs-on: ubuntu-latest     services:       postgres:         image: postgres:15
env:           POSTGRES_DB: ticketing_test           POSTGRES_USER: test           POSTGRES_PASSWORD:
test          options: >-           --health-cmd pg_isready           --health-interval 10s
--health-timeout 5s           --health-retries 5         ports:           - 5432:5432       redis:
image: redis:7         options: --health-cmd "redis-cli ping"         ports:           - 6379:6379
rabbitmq:         image: rabbitmq:3.12-management         ports:           - 5672:5672     steps:
- uses: actions/checkout@v4       - uses: actions/setup-java@v4         with:           java-version:
'17'           distribution: 'temurin'       - name: Cache Maven dependencies         uses:
actions/cache@v3         with:           path: ~/.m2           key: ${{ runner.os }}-m2-${{
hashFiles('**/pom.xml') }}       - name: Run tests         run: ./mvnw test -
Dspring.profiles.active=test         env:           SPRING_DATASOURCE_URL:
jdbc:postgresql://localhost:5432/ticketing_test           SPRING_DATASOURCE_USERNAME: test
SPRING_DATASOURCE_PASSWORD: test           SPRING_REDIS_HOST: localhost
SPRING_RABBITMQ_HOST: localhost       - name: Upload coverage report         uses: actions/upload-
artifact@v3         with:           name: jacoco-report           path: target/site/jacoco/   build:
runs-on: ubuntu-latest     needs: test     steps:       - uses: actions/checkout@v4       - uses:
actions/setup-java@v4         with: { java-version: '17', distribution: 'temurin' }       - name:
Build Docker image         run: docker build -t ticketing-platform:${{ github.sha }} .
```

Multiple confusion needs clarifications within this whole section of Docker

Optimized Dockerfile — Multi-Stage Build

- **Stage 1 (builder):** Use `eclipse-temurin:17-jdk-alpine`. Copy `.mvn/`, `mvnw`, and `pom.xml` first, then run `./mvnw dependency:go-offline -B` to cache the dependency layer (so subsequent builds skip this if `pom.xml` hasn't changed). Then copy `src/` and run `./mvnw package -DskipTests -B`.
- **Stage 2 (runtime):** Use `eclipse-temurin:17-jre-alpine` (JRE only, much smaller). Create a non-root `spring` user and group. Copy the built JAR from the builder stage. Expose port `8080`. Set the entrypoint with container-aware JVM flags: `-XX:+UseContainerSupport` and `-XX:MaxRAMPercentage=75.0`.

Code within V.1 PDF Page 31

.github/workflows/deploy.yml — Railway Deployment

- Trigger: push to `main` branch.
- Use `bervProject/railway-deploy@main` action with `railway_token: ${{ secrets.RAILWAY_TOKEN }}` and `service: ticketing-platform`.
- **Railway setup:** Create a Railway project, add PostgreSQL + Redis + RabbitMQ services (Railway plugins), copy `RAILWAY_TOKEN`, add to GitHub Secrets. Set environment variables in Railway: `STRIPE_SECRET_KEY`, `STRIPE_WEBHOOK_SECRET`, `SPRING_DATASOURCE_URL` (from Railway PostgreSQL plugin), `SPRING_REDIS_URL`, `SPRING_RABBITMQ_URL`, `SPRING_PROFILES_ACTIVE=prod`.

Evening *(1 hr)*

- Trigger a deployment. Verify: `https://{your-app}.up.railway.app/actuator/health` should return `{"status":"UP"}`.
- Update the Stripe webhook endpoint in the Stripe dashboard to: `https://{your-app}.up.railway.app/api/webhooks/stripe`.
- Git commit: `ci: add github actions ci cd pipeline and docker optimized build`.

Deliverable: GitHub Actions running on every push, Docker image built successfully, app deployed to Railway with live URL and /actuator/health returning UP.

DAY 19 — Wednesday, April 22 *Theme: Documentation + Swagger + README Completion*	5 hrs planned hours

Morning *(1 hr)*

- **System Design — "Design Eventbrite" (45 minutes):** How is Eventbrite different from Ticketmaster? (organizer-centric vs venue-centric). Free events vs paid events (no payment flow for free). Attendee check-in via QR scan API. Compare your built system vs Eventbrite's architecture. Write 3 specific trade-offs.

Afternoon *(3.5 hrs)*

Swagger / OpenAPI Documentation

- Add `springdoc-openapi-starter-webmvc-ui:2.2.0` to `pom.xml`.
- Annotate all controllers with `@Operation(summary = "...")`, `@ApiResponse(responseCode = "201")`, and `@Parameter`. Annotate all request/response DTOs with `@Schema`.
- Swagger UI available at: `http://localhost:8080/swagger-ui.html` (and the live Railway URL + `/swagger-ui.html`).

README.md — Complete Documentation

- Project title + 2-sentence description.
- Live demo URL (Railway deployment link).
- Architecture diagram (screenshot of Excalidraw diagram).
- State machine diagrams (both Booking and Event lifecycles — screenshots).
- Tech stack badges (Java, Spring Boot, PostgreSQL, Redis, RabbitMQ, Stripe, Next.js, Docker).
- Quick start: `git clone → docker-compose up → open localhost:3000`.
- Environment variables table: name, description, example value.
- Database schema (link to ERD image or embedded screenshot).
- API documentation (link to Swagger UI at `/swagger-ui.html`).

- Running tests: `./mvnw test` and coverage report command.
- Load testing: k6 run command and results screenshot.
- Project structure tree (top-level modules only).
- "What I Learned" section with 3 bullet points — written for portfolio interviewers.
- UI screenshots: at least 5 (home page, event detail, cart with countdown, confirmation with QR codes, organizer dashboard).

Evening *(1 hr)*

- Add inline code comments to all complex methods — state machine config, distributed lock service, pricing engine. Explain the **why**, not the what.
- Git commit: `docs: complete readme with architecture diagrams and api documentation`.

Deliverable: Swagger UI live at /swagger-ui.html on the Railway URL. README complete with screenshots, diagrams, and setup instructions.

DAY 20 — Thursday, April 23

Theme: Final Polish + Bug Bash + Demo Preparation

5 hrs planned hours

Morning *(1 hr)*

- **Feature walkthrough:** Write a list of every feature in the spec. Test each one manually against the deployed Railway URL. Note every bug or gap found — this is your bug bash backlog for the afternoon.

Afternoon *(3.5 hrs)*

Bug Fixes — Common Production Issues

- **CORS configuration:** If the Railway frontend cannot reach the Railway backend, add the Railway domain to `WebMvcConfigurer.addCorsMappings()`.
- **Redis connection string:** Railway's Redis URL format may differ from local `host:port` config. Switch to using `spring.data.redis.url` instead of separate host and port properties.
- **Stripe webhook URL:** Update the webhook endpoint in the Stripe dashboard to point to the Railway URL.
- **Flyway migration conflict:** If the Railway DB already has a partial schema, add `spring.flyway.baseline-on-migrate=true` to `application.yml` for the prod profile.

Frontend Polish

- Add proper error states: when the API is unreachable, show "Service temporarily unavailable" instead of a blank page.
- Add empty states: when users have no bookings, show "No bookings yet. Browse events!" with a CTA button.

- Add loading states to all async operations — every button that triggers an API call should show a loading spinner.
- Verify all form error messages are descriptive (not just "Invalid").

Final Test Run

- [ ] `./mvnw clean test` — all tests pass.
- [ ] `./mvnw jacoco:report` — coverage ≥ 80%.
- [ ] k6 load test against the Railway URL if feasible; otherwise run locally against `localhost:8080`.

Evening *(1 hr)*

- **Demo video:** Record a 3-minute Loom video demonstrating the full flow: browse events → select seats → cart countdown → Stripe payment → confirmation with QR codes → organizer dashboard showing revenue. Paste the Loom URL into the README.
- Git final commit: `chore: final polish, all tests passing, production deployment verified`.

Deliverable: Bug-free production deployment on Railway. 3-minute demo video recorded and linked in README.

DAY 21 — Friday, April 24 *Theme: Final Checkpoint + Phase 1A Closeout + Phase 1B Planning*	5 hrs planned hours

Morning *(1.5 hrs)*

- **Self-assessment:** Go through every success criteria item below. For each item NOT fully completed, write exactly what is missing and decide: fix today, or note as carry-forward with a specific completion date.

Afternoon *(3 hrs)*

Portfolio Preparation

- Add project to LinkedIn (Project section): title, description, tech stack, links to GitHub and live demo.
- Pin the GitHub repository and ensure the README looks polished with screenshots visible in the preview.
- **Resume bullet points (write 5–10):** "Designed and implemented a high-concurrency event ticketing platform handling 1,000 concurrent users with Redis distributed locking and Spring State Machine, achieving P95 response time < 500ms with zero oversell incidents."

Phase 1B Planning

- What did you NOT finish from Phase 1A? List everything honestly.
- What was the single hardest technical concept this phase? Write it down — this is an interview talking point.
- What would you do differently if starting over?
- **Phase 1B Options:**
  - **Option A — Social Media Analytics Platform:** Teaches data pipelines, batch jobs (Spring Batch), graph relationships, more Redis patterns. Builds directly on Phase 1A async processing.
  - **Option B — Real-Time Collaborative Whiteboard:** Teaches WebSockets, Spring WebFlux (reactive), CRDT concepts. Better suited for Phase 3+.
  - **Option C — E-Commerce Platform:** Teaches deeper inventory management, recommendation engine, Elasticsearch basics, order management. Reinforces payment patterns.
- **Recommendation:** Option A or C — both reinforce and extend Phase 1A patterns. Avoid Option B at this stage.

Final Code Cleanup

- Remove all TODO/FIXME comments — either implement the thing or delete the comment.
- Remove all `System.out.println()` debug statements.
- Ensure `application-local.yml` is in `.gitignore` (no secrets in the repo).
- Run `./mvnw checkstyle:check` (if configured) or perform a final manual static analysis pass.

Evening *(30 min)*

- Final git push. Tag the release: `git tag -a v1.0.0 -m "Phase 1A complete: Event Ticketing Platform"`. Push tags: `git push --tags`.
- Celebrate. You built a production-grade, payment-integrated ticketing platform in 3.5 weeks.

Deliverable: Phase 1A complete. GitHub repo polished, deployment live, portfolio updated, Phase 1B direction decided, v1.0.0 tagged.

WEEK 3 FINAL CHECKPOINT — Friday, April 24

- ☑ Multi-tier pricing (early bird, group, VIP, dynamic) working in production
- ☑ Refund logic: all 3 tiers implemented and tested with 100% branch coverage
- ☑ Waitlist: join + notify-on-seat-release working end-to-end
- ☑ QR code generation: visible on the confirmation page in the browser
- ☑ All 7 frontend pages complete and mobile responsive
- ☑ [object Object][object Object][object Object][object Object][object Object][object Object]
- ☑ k6 results: P95 < 500ms, 0 double-bookings (documented in README)

- [x] [object Object][object Object]
- [x] [object Object][object Object][object Object]
- [x] [object Object][object Object][object Object]
- [x] Complete README with architecture diagrams, screenshots, and setup instructions
- [x] Demo video recorded (Loom, 3 minutes) and linked in README
- [x] [object Object][object Object][object Object]

Week 3 Resources

- JaCoCo Maven plugin: `https://www.jacoco.org/jacoco/trunk/doc/maven.html`
- k6 documentation: `https://k6.io/docs/`
- k6 installation: `https://k6.io/docs/get-started/installation/`
- GitHub Actions documentation: `https://docs.github.com/en/actions/learn-github-actions/understanding-github-actions`
- Railway deployment: `https://railway.app`
- SpringDoc OpenAPI: `https://springdoc.org/`
- React Query optimistic updates: `https://tanstack.com/query/latest/docs/react/guides/optimistic-updates`
- System Design One — Ticketmaster: `https://systemdesign.one/ticketmaster-system-design/`
- Designing Data-Intensive Applications (DDIA) — Chapters 1–3

Phase 1A • Section 2 • April 4–24, 2026 | Target: 105–122 hours total | See Sections 4–16 for implementation guides